

# Teoría de Juegos

Marcelo Lemus

[marcelo.lemus@progcomp.cl](mailto:marcelo.lemus@progcomp.cl)



# Requisitos

Se asume familiaridad con:

- **Programación dinámica** sobre grafos (DP top-down / bottom-up en DAGs).
- **XOR y operaciones de bits**: propiedades, bit más significativo.
- **Complejidad asintótica**  $O(\cdot)$ , **función mex** (minimum excluded).
- **Grafos dirigidos**: nodos, aristas, DAG, grados de entrada/salida.
- **Aritmética modular** básica y patrones de periodicidad.
- **C++ y la STL** para la implementación del Nim.

No se requiere experiencia previa con teoría de juegos.



# Mapa de la clase



Guiados por el juego de Ada: cada concepto construye una pieza del rompecabezas.



# ¿Qué es un juego combinatorio?

Un **juego combinatorio imparcial** es un juego entre dos jugadores por turnos, información perfecta, sin azar, que siempre termina (DAG).

- ✓ Dos jugadores, turnos alternados
- ✓ Información perfecta (todo visible)
- ✓ Sin azar · Imparcial (mismas opciones)
- ✓ Siempre termina (DAG, sin ciclos)

- ✗ Ajedrez: partisano (piezas distintas)
- ✗ Póker: azar, información oculta
- ✗ Fútbol: tiempo continuo, empates
- ✓ Nim, Tatetí, juego de Ada

nodos = estados del juego · aristas = jugadas válidas  
en Ada: estados  $0, 1, 2, \dots, N$  ; desde  $x$  se mueve a  $x - 2^k$

**El que no puede mover → pierde (juego normal)**



# Posiciones W, L y D

## Ganadora (W)

quien mueve tiene estrategia para ganar

$\exists$  movimiento  $\rightarrow$  L

basta uno que mande al rival a perder

## Perdedora (L)

quien mueve pierde (contra juego óptimo)

$\forall$  movimientos  $\rightarrow$  W

todo lo que hagas le da ventaja al rival

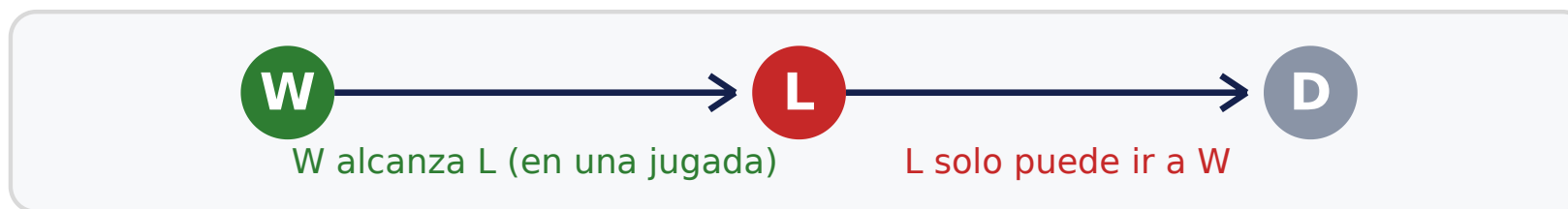
## Empate (D)

el juego puede nunca terminar

**ciclos en el grafo**

ni W ni L: se repite indefinidamente

En un DAG (sin ciclos) solo existen W y L. El empate D aparece cuando el grafo tiene ciclos.

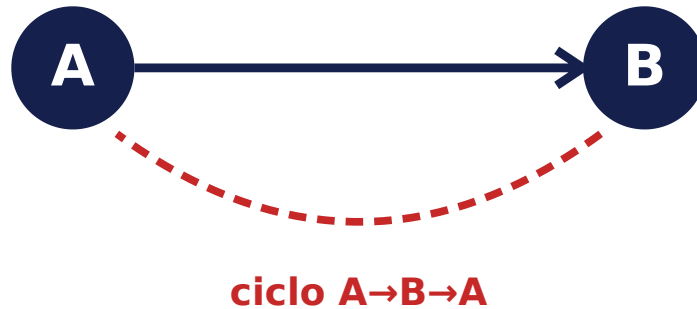




# Juegos con ciclos: ¿de dónde sale D?

Cuando el grafo de juego **no es un DAG** (tiene ciclos), aparece un tercer tipo de posición: el empate *D*.

**¿Por qué un ciclo produce empate?**



**Ambos racionalmente → empate (D)**

## Análisis:

- Ambos jugadores pueden quedarse en el ciclo indefinidamente
- Ninguno quiere salir porque perdería

En competencias, muchos juegos en grafos dirigidos tienen ciclos. El análisis W/L no basta: necesitamos el algoritmo de **3 colores**.



# Algoritmo de 3 colores (W, L, D)

## Algoritmo para etiquetar grafos dirigidos con ciclos

### 1 · Terminales $\rightarrow$ L

Nodos sin aristas salientes  
Siempre son perdedores  
(no se puede mover)

### 2 · $\exists$ hijo L $\rightarrow$ W

Si existe un movimiento  
que lleva a una posición L,  
esta posición es W

### 3 · $\forall$ hijos W $\rightarrow$ L

Si todos los movimientos  
van a posiciones W,  
esta posición es L

### 4 · Lo que no es W ni L $\rightarrow$ D (empate)

BFS inverso: empezar desde terminales, propagar W/L, el resto es D

**Complejidad:  $O(V + E)$  con BFS en el grafo reverso**



# Ejemplo: juego con empates

## Grafo de juego con un ciclo — resolvamos paso a paso



**Con ciclos, W/L no alcanza: aparece D (empate)**

En problemas con grafos cíclicos, la presencia de un  $D$  alcanzable cambia completamente el análisis.





# Propiedad universal

Si  $\{W, L\}$  cumple estas 3 reglas  $\rightarrow$  es la partición correcta del juego

## 1 · Terminales

son L (no hay movimientos posibles)

## 2 · W no terminal

$\exists$  jugada de W a L  
(puedo mandarlo a perder)

## 3 · L no terminal

$\forall$  jugada de L va a W  
(todo le da ventaja al rival)

**equivalente a la DP**



pero sin calcular todo el DAG

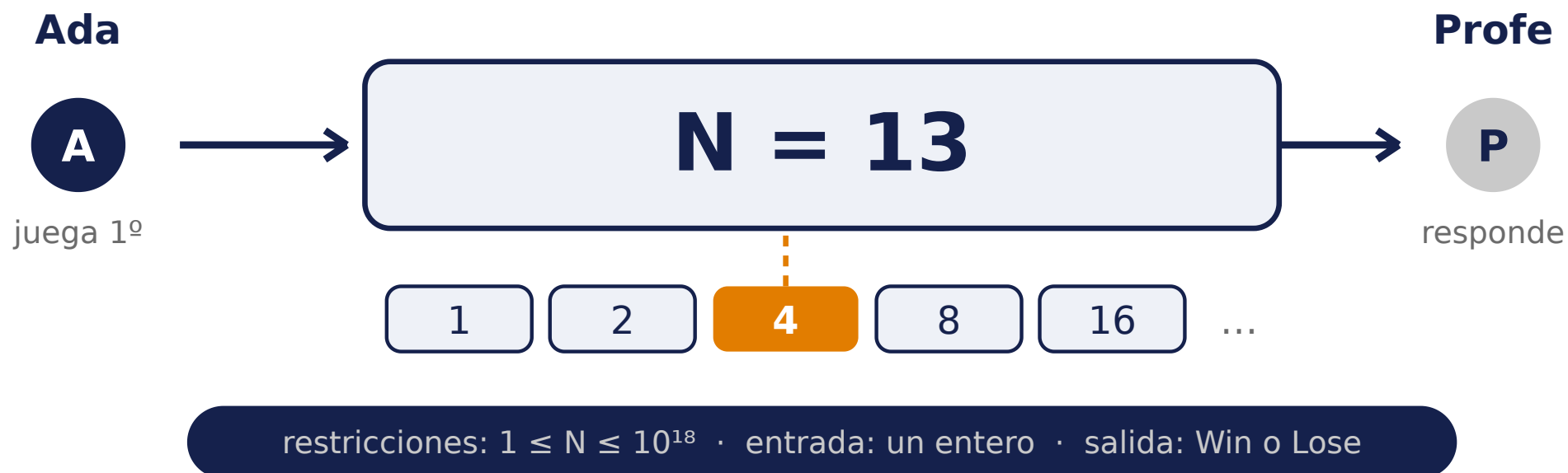
Basta conjeturar  $\{W, L\}$  y verificar las reglas. Es la **herramienta principal** para demostrar estrategias.



# El juego de Ada

Ada y su profesor de Física juegan: comienzan con un entero positivo  $N$  en el tablero. Ada juega primero.

En cada turno, el jugador **resta una potencia de 2** ( $1, 2, 4, 8, 16, \dots$ ) menor o igual al número actual. Quien escribe  $0$  **gana**.



¿Tiene Ada una estrategia ganadora?



# Ada: primeras ideas ( $N = 0 \dots 6$ )

¿Gana o pierde Ada para cada  $N$ ?

| N   | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|-----|---|---|---|---|---|---|---|
| W/L | L | W | W | L | W | W | L |

Razonamiento para cada valor:

$N = 0$ : terminal, no hay movimientos  $\rightarrow L$

$N = 1$ : restar 1  $\rightarrow 0 (L) \rightarrow W$

$N = 2$ : restar 2  $\rightarrow 0 (L) \rightarrow W$

$N = 3$ :  $1 \rightarrow 2 (W)$  y  $2 \rightarrow 1 (W) \rightarrow L$

$N = 4$ :  $1 \rightarrow 3 (L)$ ,  $2 \rightarrow 2 (W)$ ,  $4 \rightarrow 0 (L) \rightarrow \exists L \rightarrow W$

$N = 5$ :  $2 \rightarrow 3 (L)$ ,  $1 \rightarrow 4 (W) \rightarrow \exists L \rightarrow W$

$N = 6$ :  $1 \rightarrow 5 (W)$ ,  $2 \rightarrow 4 (W)$ ,  $4 \rightarrow 2 (W) \rightarrow L$

Sabemos que  $W$  = "existe movimiento a  $L$ " y  $L$  = "todos los movimientos van a  $W$ ".



# Ada: más valores y conjetura

| N:   | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 |
|------|---|---|---|----|----|----|----|----|----|
| W/L: | W | W | L | W  | W  | L  | W  | W  | L  |

el bloque W, W, L se repite cada 3



**Conjetura:  $W \iff N \bmod 3 \neq 0$**

Ada pierde exactamente cuando N es múltiplo de 3

Completando la tabla: W, W, L cada tres. Pero... ¿cómo demostrarlo? Necesitamos la teoría.



# DP sobre el grafo de juego

Resolvemos el juego con DP: de las hojas hacia la raíz

## 1 · Terminales

sin movimientos  $\rightarrow$  L  
(quien no puede jugar pierde)

## 2 · Nodo con hijos L

$\exists$  hijo marcado L  
 $\rightarrow$  este nodo es W

## 3 · Nodo sin hijos L

$\forall$  hijos son W  
 $\rightarrow$  este nodo es L

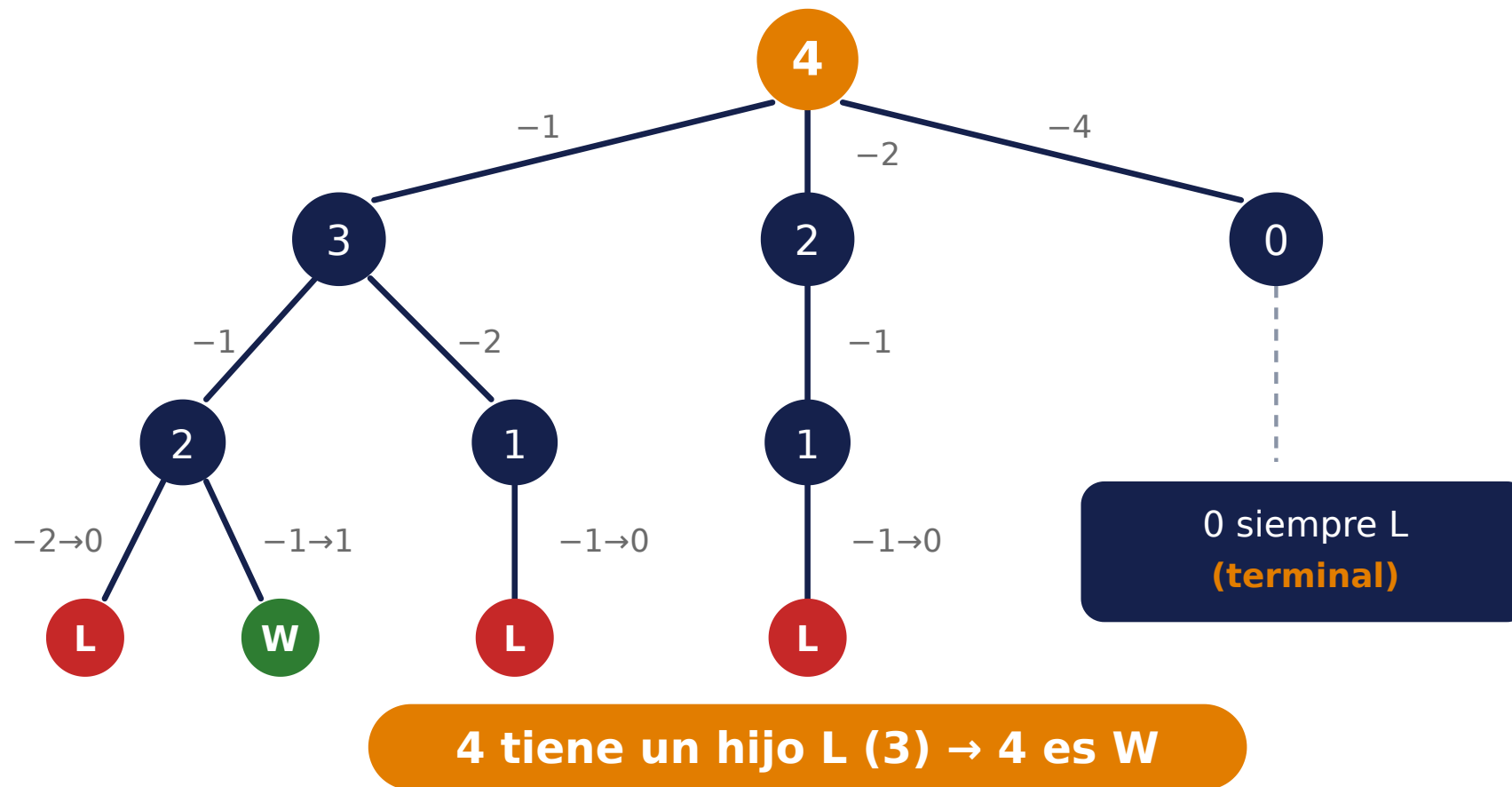
Ejemplo: fragmento del juego de Ada



Algoritmo: recorrer en **orden topológico**. Terminales  $\rightarrow$  L. Luego, W si  $\exists$  hijo L, L si  $\forall$  hijos W.



# DP: árbol completo de Ada para $N = 4$





# Ada: tabla completa de DP

Calculamos W/L para  $N = 0..11$  con la DP

| N   | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
|-----|---|---|---|---|---|---|---|---|---|---|----|----|
| W/L | L | W | W | L | W | W | L | W | W | L | W  | W  |

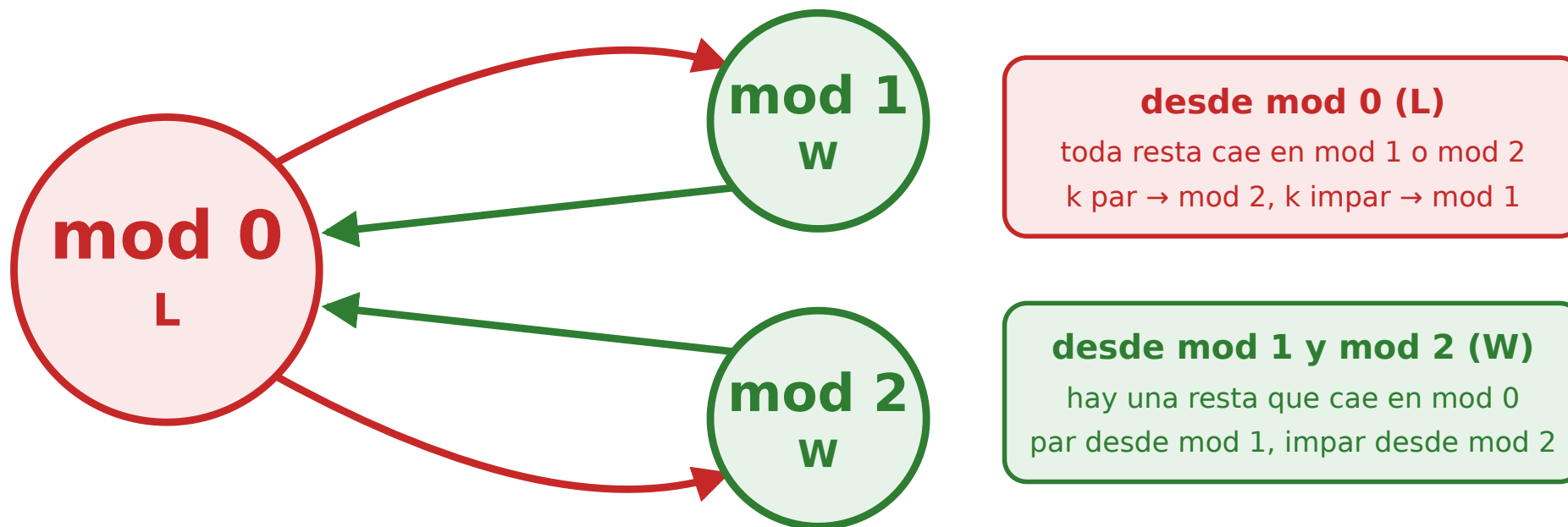
**Conjetura:  $W \iff N \bmod 3 \neq 0$**

Demostremos con la propiedad universal: solo hay que verificar las 3 reglas



# Ada: demostración del patrón mod 3

Propiedad clave:  $2^k \bmod 3 = 1$  si  $k$  es par,  $= 2$  si  $k$  es impar



Verificación de la propiedad universal completa — no hace falta más





# Ada: solución final

$N \bmod 3 = 0 \implies \text{Lose,}$

$N \bmod 3 \neq 0 \implies \text{Win}$

**$N \bmod 3 \neq 0$**

Ada gana

estrategia: restar  $2^k$   
para volver a mod 0

$k$  par si mod 1,  $k$  impar si mod 2

**$N \bmod 3 = 0$**

Ada pierde

cualquier resta va a  
mod 1 o mod 2 (W)

el profe responde a mod 0

**Complejidad:**  $O(1)$ . Con  $N \leq 10^{18}$  cabe en `long long`. Respuesta:

```
N % 3 == 0 ? "Lose" : "Win".
```



# Juegos de sustracción

Un **juego de sustracción** se define por un conjunto  $S$  de restas permitidas. Ada es el caso  $S = \{1, 2, 4, 8, 16, \dots\}$ .

## Reglas generales

$S = \{s_1, s_2, \dots\}$  (finito o no)

Desde  $x$ : mover a  $x - s$  (si  $s \in S, s \leq x$ )

**W/L se calcula con DP hacia atrás**

## Teorema de periodicidad

Si  $S$  es finito, el patrón W/L es eventualmente periódico.

A lo más  $2^{\max(S)}$  estados posibles

Ada tiene  $S$  infinito, pero el patrón es periódico igual. ¿Por qué?

**$2^k \bmod 3$  alterna entre 1 y 2  $\rightarrow$  estructura cada 3**

Aunque  $S$  es infinito, la estructura módulo 3 de las potencias de 2 nos da la demostración.



| N   | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|-----|---|---|---|---|---|---|---|---|---|---|----|----|----|
| W/L | L | W | L | W | W | W | W | L | W | L | W  | W  | W  |

primer período
se repite igual

Con  $S$  finito, el patrón W/L siempre termina siendo periódico. Con  $S$  infinito (Ada), puede ser periódico también.



# Sustracción: cómo buscar periodicidad

Cuando las constraints son grandes y  $S$  es finito, la DP directa no alcanza. Pero sabemos que el patrón es **eventualmente periódico**.

## Estrategia práctica

1. Calcular W/L hasta  $\sim 1000$  (o más si cabe)
2. Buscar repetición de un bloque de largo  $L$
3. Verificar que el bloque se repite  $\geq 3$  veces

$L \leq 2^{\max(S)}$  en el peor caso

Si no encontramos período con  $\sim 1000$  valores, probablemente el problema espera DP directa o Grundy.

## Ejemplo: $S = \{2, 5\}$

W/L para  $N = 0..13$ :

L, L, W, L, L, W, W, L, W, L, W, W, L, W

Período 7 a partir de  $N = 4$

**En CP: si ves un patrón, úsalo ✓**



# Sustracción con límite: Nim(k)

Un caso especial importante: solo se pueden quitar **entre 1 y k** piedras.

$$S = \{1, 2, \dots, k\}.$$

$$g(x) = x \bmod (k + 1) \cdot W \iff x \bmod (k+1) \neq 0$$

**k = 1 (solo quitar 1)**

$$S = \{1\} \rightarrow g(x) = x \bmod 2$$

$$W \iff x \text{ impar} \cdot L \iff x \text{ par}$$

Trivial: gana el primero si N es impar

**k = 3 (quitar 1, 2 o 3)**

$$S = \{1, 2, 3\} \rightarrow g(x) = x \bmod 4$$

$$W \iff x \bmod 4 \neq 0$$

Posiciones L: 0, 4, 8, 12, ...

Cuando  $k \rightarrow \infty$  (remoción libre)  $\rightarrow$  Nim normal  $\rightarrow g(x) = x$   
Nim(k) es el puente natural entre juegos de sustracción y Nim

# El Nim

1 · Estados W/L/D

2 · Sustracción

**3 · El Nim**

4 · S-Grundy



# El Nim: reglas

$n$  pilas de piedras. En cada turno, elegir **una** pila y quitar **cualquier cantidad positiva**.  
Quien toma la última piedra **gana**.



A diferencia de Ada, aquí puedo **vaciar la pila entera** de una sola jugada.



# Nim: la estrategia XOR

$$X = x_1 \oplus x_2 \oplus \dots \oplus x_n$$

**$X = 0 \iff L$  (perdedora) ·  $X \neq 0 \iff W$  (ganadora)**

Ejemplo:  $5 \oplus 3 \oplus 4 \oplus 1$

5 =  $0101_2$

3 =  $0011_2$

4 =  $0100_2$

1 =  $0001_2$

$$X = 0011_2 = 3$$

**$X \neq 0 \rightarrow W$  (gana el primero)**

**$X = 0 \rightarrow L$  (pierde el primero)**

Propiedad sorprendente de la teoría de juegos. ¿Por qué funciona?





# Nim: demostración con propiedad universal

## Caso 1: $X = 0 \rightarrow L$

Quitar  $k$  piedras de la pila  $i$ :

$x_i$  cambia a  $x'_i \neq x_i \rightarrow$  nuevo  $XOR = x'_i \oplus x_i \neq 0$

**todo movimiento rompe el  $XOR = 0$   
 $\rightarrow W$  para el rival**

## Caso 2: $X \neq 0 \rightarrow W$

Sea  $p = \text{MSB de } X$ .  $\exists$  pila  $i$  con bit  $p$  en  $x_i$ .

Restar:  $x_i \rightarrow x_i \oplus X$  ( $< x_i$ : se apaga el bit  $p$ )

**nuevo  $XOR = (x_i \oplus X) \oplus (X \oplus x_i) = 0$   
 $\rightarrow$  deja al rival en  $L$  ✓**

## Ilustración del bit más significativo

$X = \dots 0 \mathbf{1} 0 1 1 0$        $p = \text{el bit más significativo de } X$

$x_i = \dots \mathbf{1} \dots$        $x_i$  tiene ese bit en 1

apagar el bit  $p$  garantiza  $x_i \oplus X < x_i \rightarrow$  puedo restar esa cantidad

**Propiedad universal verificada:** terminal  $X = 0$ , de  $X \neq 0 \exists$  jugada a  $X = 0$ , de  $X = 0 \forall$  jugada a  $X \neq 0$ .



# Nim: implementación

```
#include <iostream>
using namespace std;

int main() {
    int t; cin >> t;
    while (t--) {
        int n; cin >> n;
        int x = 0;
        for (int i = 0; i < n; i++) {
            int pile; cin >> pile;
            x ^= pile;
        }
        cout << (x ? "first" : "second") << '\n';
    }
}
```

**Complejidad:**  $O(n)$  por caso de prueba. Sin memoria extra.



# Suma de juegos

Dos juegos imparciales  $A$  y  $B$  se pueden jugar **en simultáneo**: en cada turno, el jugador elige **uno** y hace una jugada válida en él.



**¿Cómo resolvemos  $A + B$  sin recalcular desde cero?**

**Sprague-Grundy: reducir cada juego a un número**

# Sprague-Grundy

1 · Estados W/L/D

2 · Sustracción

3 · El Nim

4 · **S-Grundy**

# Grundy numbers: definición y mex

A cada estado de un juego imparcial le asignamos un número: el **Grundy number**  $g$  (o *nimber*).

$$g(\text{estado}) = \text{mex}\{ g(\text{hijo}_1), g(\text{hijo}_2), \dots \}$$

$\text{mex}(S) = \text{menor entero } \geq 0 \text{ que NO está en } S$

**Ejemplo:  $\text{mex}(\{0, 2, 3\}) = ?$**



en rojo: los que sí están en el conjunto

**mex = 1**

0 está en el conjunto, 1 es el primero que no  $\rightarrow \text{mex} = 1$

Propiedades:  $g = 0 \iff \text{posición L}; g > 0 \iff \text{posición W}.$

# Grundy: calculamos $g(n)$ para Ada

$$g(n) = \text{mex}\{ g(n-1), g(n-2), g(n-4), g(n-8), \dots \}$$

Calculemos paso a paso:

$$g(0) = \text{mex}\{\emptyset\} = \mathbf{0}$$

$$g(1) = \text{mex}\{g(0)\} = \text{mex}\{0\} = \mathbf{1}$$

$$g(2) = \text{mex}\{g(1), g(0)\} = \text{mex}\{1, 0\} = \mathbf{2}$$

$$g(3) = \text{mex}\{g(2), g(1)\} = \text{mex}\{2, 1\} = \mathbf{0}$$

$$g(4) = \text{mex}\{g(3), g(2), g(0)\} = \text{mex}\{0, 2, 0\} = \mathbf{1}$$

$$g(5) = \text{mex}\{g(4), g(3), g(1)\} = \text{mex}\{1, 0, 1\} = \mathbf{2}$$

$$g(6) = \text{mex}\{g(5), g(4), g(2)\} = \text{mex}\{2, 1, 2\} = \mathbf{0}$$

| n | g(n) |
|---|------|
| 0 | 0    |
| 1 | 1    |
| 2 | 2    |
| 3 | 0    |
| 4 | 1    |
| 5 | 2    |
| 6 | 0    |

$$g(n) = n \bmod 3 \cdot \text{igual que W/L}$$

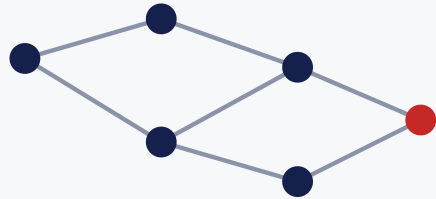
$g = 0$  coincide con  $L$ ,  $g > 0$  con  $W$ . Pero Grundy da **más**: el número exacto, no solo W/L.



# Teorema de Sprague-Grundy

## Teorema — Sprague (1935), Grundy (1939)

Todo juego combinatorio imparcial  $G$  es equivalente a una pila de Nim de tamaño  $g(G)$



juego original  $G$



equivalen

$g(G)$  piedras



pila de Nim

$$g(A + B) = g(A) \oplus g(B)$$

Con esto cualquier juego imparcial se resuelve con: calcular  $g$  con mex  $\rightarrow$  XOR de todos los  $g \rightarrow$  ¿XOR = 0?

# Nim como caso especial de Grundy

¿Cuál es el Grundy de una pila de Nim de tamaño  $x$ ? ¡Es simplemente  $x$ !

$$g(x) = \text{mex}\{ g(0), g(1), \dots, g(x-1) \}$$

Por inducción:  $g(0)=0, g(1)=1, \dots, g(x-1)=x-1$   
 $\rightarrow \text{mex}\{0, 1, 2, \dots, x-1\} = x$

$$g(\text{pila de } x) = x \cdot \text{XOR de } g = \text{XOR de pilas} \checkmark$$

**El Nim ES el caso base de Grundy**

## Nim normal

$g$  de cada pila = tamaño  
 $\text{XOR de } g \rightarrow \text{¿0?} \rightarrow L : W$

## Grundy general

$g$  de cada sub juego = mex  
 $\text{XOR de } g \rightarrow \text{¿0?} \rightarrow L : W$

Nim es lo que pasa cuando cada sub juego **ya está en su forma canónica**:  $g(x) = x$ .  
Grundy generaliza esto a cualquier juego imparcial.



**Todo juego imparcial es equivalente a UN montón de Nim**

**$g(\text{estado}) = \text{tamaño de la pila de Nim equivalente}$**

**Nim:  $g(x) = x$**

- Una pila → Grundy = tamaño
- XOR de pilas = XOR de g's

**Grundy:  $g = \text{mex}$**

- Calculamos g con mex recursivo
- XOR de g's = resultado final

$$g(A + B) = g(A) \oplus g(B)$$

La suma de juegos se reduce a XOR de sus Grundy numbers



# Crosses-Crosses: reglas

Tira de  $1 \times n$  casillas. En cada turno, el jugador **pone una cruz** en una casilla vacía, con la restricción de que **nunca puede haber dos cruces adyacentes** (vecinas izquierda/derecha).

**$n = 7$ , una cruz ya puesta en la posición 3**



**X prohibidos:**



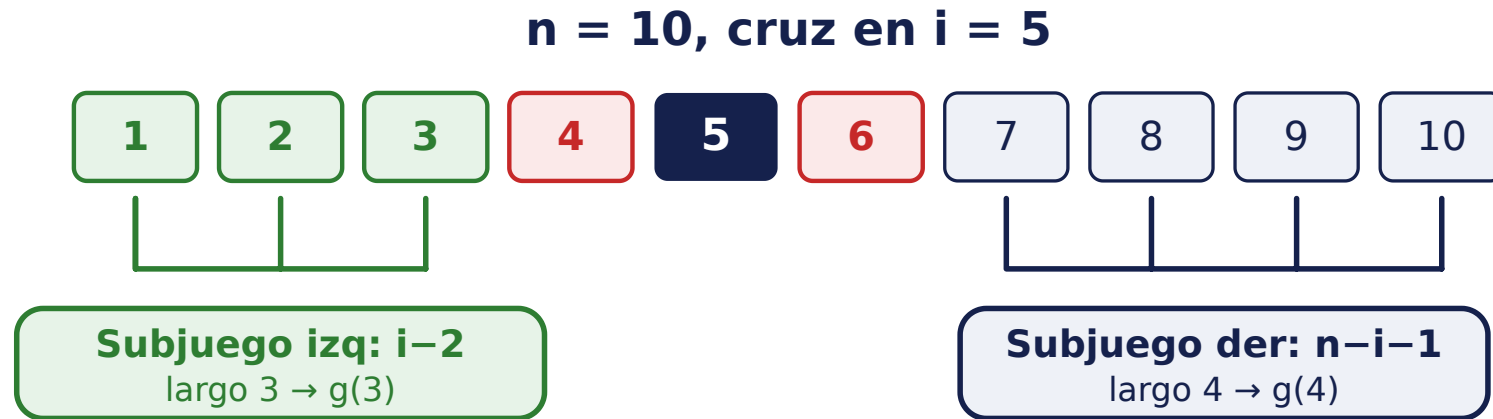
→ la cruz destruye su casilla y las 2 vecinas

**Cada jugada parte el juego en dos tiras independientes**

Juego imparcial, DAG, sin azar. Quien no puede poner cruz → **pierde**.

# Crosses-Crosses: descomposición

Al poner cruz en la posición  $i$ , se inutilizan  $i - 1$ ,  $i$ ,  $i + 1$ . El juego se parte en **dos subjuegos independientes** de largo  $i - 2$  y  $n - i - 1$ .



**$g(10)$  necesita  $\text{mex}\{ \dots, g(3) \oplus g(4), \dots \}$**

Cuando una jugada PARTE el juego en varios subjuegos,  
¡el Grundy del estado resultante es el XOR de los Grundy de cada parte!

El XOR de Grundy aparece **dentro del mex** cuando hay splitting. Es la firma de Grundy.

$$g(n) = \text{mex}(\{g(n-2)\} \cup \{g(i-2) \oplus g(n-i-1) \mid 2 \leq i \leq n-1\})$$

$g(x) = 0$  para  $x \leq 0$  (tira vacía o inexistente)

$$g(0) = \text{mex}\{\emptyset\} = 0$$

$$g(1) = \text{mex}\{g(-1)\} = \text{mex}\{0\} = 1$$

$$g(2) = \text{mex}\{g(0)\} = \text{mex}\{0\} = 1$$

$$g(3) = \text{mex}\{g(1), g(0) \oplus g(0)\} = \text{mex}\{1, 0\} = 2$$

$$g(4) = \text{mex}\{g(2), g(0) \oplus g(1), g(1) \oplus g(0)\} = \text{mex}\{1, 1, 1\} = 0$$

$$g(5) = \text{mex}\{g(3), g(0) \oplus g(2), g(1) \oplus g(1), g(2) \oplus g(0)\} = \text{mex}\{2, 1, 0, 1\} = 3$$

$$g(6) = \text{mex}\{g(4), g(0) \oplus g(3), g(1) \oplus g(2), g(2) \oplus g(1), g(3) \oplus g(0)\} = \text{mex}\{0, 2, 0, 0, 2\} = 1$$

| n | g(n) |
|---|------|
| 0 | 0    |
| 1 | 1    |
| 2 | 1    |
| 3 | 2    |
| 4 | 0    |
| 5 | 3    |
| 6 | 1    |

$$g(4) = 0 \rightarrow L \cdot g(1) = g(2) = 1, g(3) = 2 \rightarrow W$$

```
const int MAXN = 200;
int g[MAXN+1];
bool seen[256];

void precompute() {
    g[0] = g[1] = 0;
    for (int n = 2; n <= MAXN; n++) {
        memset(seen, 0, sizeof(seen));
        seen[g[n-2]] = true;           // borde
        for (int i = 2; i <= n-1; i++)
            seen[g[i-2] ^ g[n-i-1]] = true; // splitting
        for (int x = 0; ; x++)
            if (!seen[x]) { g[n] = x; break; }
    }
}
```

**Complejidad:**  $O(n^2)$  para precalcular hasta  $n$ . Para  $n$  grande:  $g(n)$  tiene período 34 a partir de  $n = 52$ .

Precalcular hasta  $n = 85$ , luego  $g(n) = g(n-34)$  para  $n \geq 52$ . Con esto resolvemos  $n$  arbitrariamente grande en  $O(1)$ .



# Patrones en tablas de Grundy

Muchas veces no necesitamos calcular  $g$  para todo  $n$  enorme: los valores de Grundy suelen ser **periódicos** o seguir un patrón simple.

## Estrategia general

1. Calcular  $g(0) \dots g(m)$  para  $m$  moderado
2. Buscar repetición en la secuencia
3. Verificar que el patrón se mantiene
4. Demostrar por inducción si es necesario

### Tabla de referencia rápida:

Nim:  $g(x) = x$  (sin período)

Bounded Nim( $k$ ):  $g(x) = x \bmod (k+1)$

Sustracción  $S$  finito: período  $\leq 2^{\max(S)}$

## Ejemplos de períodos

Ada:  $S = \text{potencias de } 2 \rightarrow g(n) = n \bmod 3$

$S = \{1, 3, 4\} \rightarrow \text{período } 7$

Crosses-Crosses  $\rightarrow \text{período } 34$

Kayles  $\rightarrow \text{período } 12$

### ¡Ojo! No todo es periódico

El Grundy's Game (partir un montón en dos desiguales) sigue sin patrón conocido

En competencias, si las constraints son grandes y ves un patrón en los primeros 100-200 valores, **confía en él**.



## S-Nim (Kattis)

Nim con  $S$  = conjunto de restas permitidas

Ej:  $S = \{2, 3, 5\}$   
pilas: 10, 7, 4, 3

**Precalcular  $g(0) \dots g(\text{max pila})$**

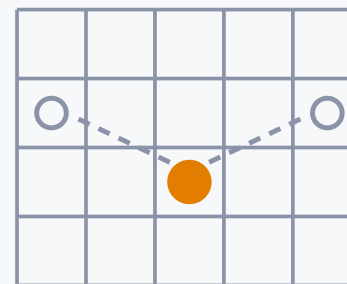
$g(i) = \text{mex}\{g(i-s) \mid s \in S, s \leq i\}$

**XOR de  $g$  de cada pila**

**$O(P \cdot K + M \cdot L)$ ,  $P = \text{max pila}$**

## Chessboard Game (Hackerrank)

Tablero  $15 \times 15$  con monedas  
mover como caballo (↖ ↗)



cada moneda = subjuego

**$g(r,c) = \text{mex} \cdot \text{XOR de todos}$**

Patrón universal: **descomponer en subjuegos, calcular  $g$  con mex, combinar con XOR.**



# Problemas recomendados

| Problema                     | Fuente         | Dificultad | Técnica                |
|------------------------------|----------------|------------|------------------------|
| Nim Game I                   | CSES 1730      | ★          | Nim, XOR               |
| Nim Game II                  | CSES 1098      | ★★         | Nim, misère            |
| Ada and Game of Divisors     | CF 1534D       | ★★         | Sustracción, patrones  |
| S-Nim                        | Kattis         | ★★         | Grundy, mex            |
| Crosses-Crosses              | Clásico        | ★★         | Grundy, splitting, XOR |
| Tower Breakers, Revisited!   | Hackerrank     | ★★         | Grundy, bounded Nim    |
| Chessboard Game, Again!      | Hackerrank     | ★★★        | Grundy, descomposición |
| Game of Stones               | CF 768E        | ★★★★       | Grundy, DP             |
| Interval Game 2              | AtCoder ABC206 | ★★★★       | Grundy, intervalos     |
| Apple Tree                   | CF 812E        | ★★★★       | Nimbers, árbol         |
| Game on Tree                 | AtCoder AGC017 | ★★★★       | Nimbers, árbol         |
| Marbles (Brazil Subregional) | CF 101908B     | ★★★★       | Grundy, patrones       |
| Strange Nim                  | AtCoder ARC091 | ★★★★★      | Grundy, patrones       |
| Arpa & Game                  | CF 850C        | ★★★★★      | Nimbers, DP            |

Todos se resuelven calculando Grundy con mex y combinar con XOR.



**¡Fin!**